

Block 4 – Session 1

Network Virtualization

Arquitectura de Xarxes

Universitat Pompeu Fabra – 2025/2026

1 Introduction to Network Virtualization

2 Virtualization Concepts

3 Virtual Network Infrastructure

4 Network Isolation and Multi-Tenancy

5 References

6 Session Summary

Outline

- 1 Introduction to Network Virtualization
- 2 Virtualization Concepts
- 3 Virtual Network Infrastructure
- 4 Network Isolation and Multi-Tenancy
- 5 References
- 6 Session Summary

What If One Machine Could Do the Job of Ten?

What is the Cloud and how is it formed?
How do networks work in the cloud?

Learning objectives:

- 1 Understand why virtualization exists and what problems it solves
- 2 Distinguish VMs, hypervisors, and containers
- 3 Explain how networks are virtualized (vNICs, vSwitches, vRouters)

The Problem with Physical Infrastructure

- Traditional model: **one server = one application**
- Typical server utilization: only **10–15%** of capacity
- Adding capacity means buying, racking, cabling → **weeks or months**
- Hardware failure takes down the entire service
- No way to scale down when demand drops

Key insight: most servers sit idle most of the time. We're paying for hardware we barely use.

Resource Consolidation

- **Consolidation** = run multiple workloads on fewer physical machines
- Benefits:
 - Higher utilization: **60–80%** instead of 10–15%
 - Lower energy, cooling, and physical space costs
 - Simpler infrastructure management
- Trade-off: shared hardware requires **strong isolation** between workloads

Analogy: instead of one person per house, we build apartment buildings – shared structure, private spaces.

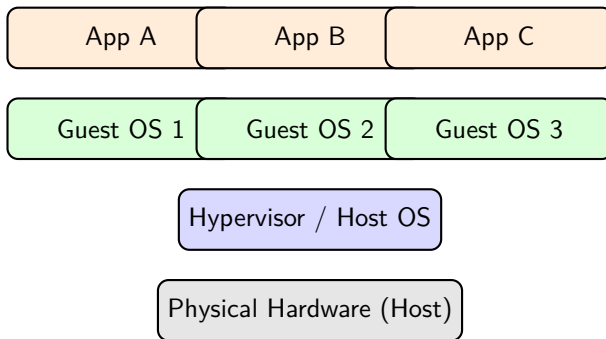
Scalability and Elasticity

- **Scalability**: ability to grow resources as demand increases
- **Elasticity**: ability to grow *and shrink* automatically
- Physical infrastructure provides **neither** in practice:
 - Can't add servers in minutes
 - Can't return servers when demand drops
- Virtualization enables both → foundation of **cloud computing**

Outline

- 1 Introduction to Network Virtualization
- 2 Virtualization Concepts**
- 3 Virtual Network Infrastructure
- 4 Network Isolation and Multi-Tenancy
- 5 References
- 6 Session Summary

Host and Guest Systems



- **Host:** physical machine providing CPU, RAM, disk, NIC
- **Guest:** virtual machine running its own OS on the host
- Each guest believes it has **dedicated hardware** (abstraction)

Source: Popek & Goldberg, "Formal Requirements for Virtualizable Third Generation Architectures," *CACM*, 1974.

Virtual Machines (VMs)

- A VM is a **software emulation** of a complete computer
- Has its own: virtual CPU, RAM, disk, network interfaces
- Runs a **full operating system** (Linux, Windows, etc.)
- Completely **isolated** from other VMs on the same host

Key properties:

- **Encapsulation**: entire VM state stored as files → easy to copy, move, backup
- **Hardware independence**: VM doesn't care about underlying hardware
- **Snapshotting**: save and restore VM state at any point in time

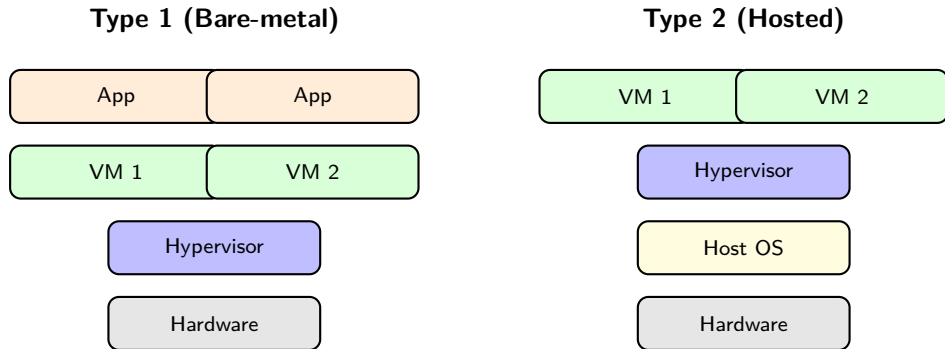
Hypervisors – The Key Enabler

- A **hypervisor** (or VMM) manages and allocates physical resources to VMs
- Two types:

	Type 1 (Bare-metal)	Type 2 (Hosted)
Runs on	Directly on hardware	On top of a host OS
Performance	Near-native	Some overhead
Use case	Data centers, production	Development, testing
Examples	VMware ESXi, KVM, Hyper-V	VirtualBox, VMware Workstation

Source: VMware, "Understanding Full Virtualization, Paravirtualization, and Hardware Assist," 2007; Barham et al., "Xen and the Art of Virtualization," *SOSP*, 2003.

Type 1 vs Type 2 – Visual Comparison



- Type 1: hypervisor **replaces** the OS → less overhead, used in production
- Type 2: hypervisor runs **as an application** → easier to set up, used for dev/testing

VMs vs Containers – Overview

	Virtual Machines	Containers
Isolation	Full OS per VM (hardware-level)	Shared kernel (namespace/cgroup)
Size	GBs (full OS image)	MBs (app + dependencies only)
Boot time	Minutes	Seconds
Overhead	Higher (full OS running)	Lower (shared kernel)
Security	Strong (hardware isolation)	Good (kernel-level isolation)

Deep dive into containers in Block 5. For now: VMs = heavy isolation, containers = lightweight isolation.
Docker, Inc., “Docker Overview,” docs.docker.com.

When to Use VMs vs Containers

- **Use VMs when:**
 - You need to run **different operating systems** (Linux + Windows)
 - Strong **security isolation** is critical (e.g., different tenants)
 - Legacy applications that require a specific OS environment
- **Use containers when:**
 - Running **many instances** of similar applications
 - **Fast startup** and scaling is important
 - **Microservices** architecture (each service in its own container)
- **In practice:** most organizations use **both** together
 - Containers run inside VMs for an extra layer of isolation (defense-in-depth)

Outline

- 1 Introduction to Network Virtualization
- 2 Virtualization Concepts
- 3 Virtual Network Infrastructure**
- 4 Network Isolation and Multi-Tenancy
- 5 References
- 6 Session Summary

Virtual Network Interfaces (vNICs)

- A **vNIC** is a software-emulated network interface card
- From the guest's perspective: behaves exactly like a real NIC
- Each VM gets **one or more vNICs**

Key characteristics:

- Has its own **MAC address** (software-assigned, not burned-in)
- Has its own **IP address** configuration
- Connected to a **virtual switch** on the host

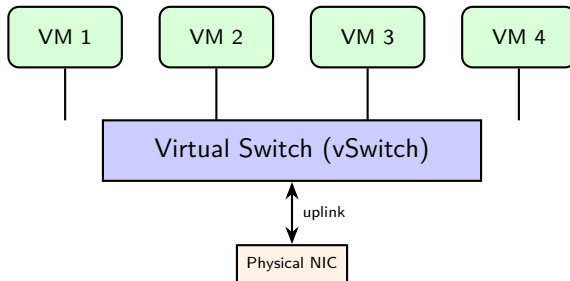
MAC Addressing in Virtual Environments

- Physical NICs: MAC **burned into hardware** (OUI + device ID)
- Virtual NICs: MAC **generated by the hypervisor**
- Common vendor prefixes identify the platform:

MAC Prefix	Platform
00:50:56	VMware
52:54:00	KVM / QEMU
00:15:5D	Microsoft Hyper-V

- Risk: MAC collisions if addresses aren't managed properly
- Hypervisors ensure **uniqueness** within the same host
- Multiple vNICs per VM enable **multi-network** connectivity

Virtual Switching



- Connects VMs on the same host – works like a **physical L2 switch**
- Maintains **MAC address table**, forwards frames by destination MAC
- Connected to physical NIC via **uplink** for external traffic

Virtual Switch Features

- Supports **VLANs** for traffic segmentation between VMs
 - Same 802.1Q from Block 3, now inside the hypervisor
- Types of virtual switches:
 - **Standard vSwitch**: basic, built into the hypervisor
 - **Distributed vSwitch**: spans multiple hosts, centralized management
 - **Open vSwitch (OVS)**: open-source, programmable, supports SDN

Source: Pfaff et al., "The Design and Implementation of Open vSwitch," *NSDI*, 2015.

Bridging Modes

Three ways to connect a VM to the outside world:

Mode	VM visibility	External access?	Use case
Bridged	Same subnet as host	Yes (direct)	Production servers
NAT	Hidden behind host IP	Outbound only	Dev/testing
Host-only	Only sees the host	No	Isolated labs

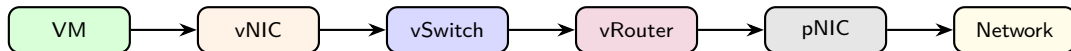
- **Bridged:** VM appears directly on the physical network
- **NAT:** VM traffic translated through the host's IP (same NAT from Block 3)
- **Host-only:** completely isolated from external networks

Virtual Routing and NAT

- A **virtual router** forwards traffic between virtual subnets
- Same function as a physical router, implemented **in software**
- Handles:
 - Routing between VMs on **different subnets**
 - **NAT** for VMs accessing external networks
 - Acting as **default gateway** for VMs

Same NAT concept from Block 3 – now managed by the hypervisor instead of a physical device.

Packet Flow – Virtual to Physical



- 1 VM generates packet with **virtual MAC/IP** as source
- 2 vNIC passes frame to **vSwitch** (L2 forwarding)
- 3 If destination is another subnet → **vRouter** (L3 forwarding)
- 4 vRouter may apply **NAT** (replace private IP with host IP)
- 5 Frame exits through **physical NIC** to the external network

Outline

- 1 Introduction to Network Virtualization
- 2 Virtualization Concepts
- 3 Virtual Network Infrastructure
- 4 Network Isolation and Multi-Tenancy**
- 5 References
- 6 Session Summary

Multi-Tenant Architectures

- **Multi-tenancy**: multiple independent users (**tenants**) share the same infrastructure
- Examples: cloud providers, enterprise data centers, university IT
- Each tenant expects:
 - **Performance isolation**: my traffic is not affected by yours
 - **Security isolation**: you cannot see or access my data
 - **Address independence**: we can both use 10.0.0.0/24

The Isolation Challenge

- Physical networks are **shared by default** – no isolation
- Without isolation mechanisms:
 - **Broadcast storms** affect everyone
 - **ARP spoofing** can intercept traffic between tenants
 - **IP address conflicts** between tenants using the same ranges
 - No **performance guarantees** (one tenant can starve others)
- Solution: **logical isolation** mechanisms at L2 and above

VLANs in Virtual Environments

- Same **802.1Q** concept from Block 3, now applied to virtual switches
- Hypervisors assign VMs to specific VLANs via **vSwitch port groups**
- Traffic tagged with VLAN IDs – tenants separated at L2

Limitation: VLAN ID is 12 bits → max **4,096 VLANs**

Problem: 4,096 VLANs is **not enough** for a large cloud provider with thousands of tenants.

Source: IEEE 802.1Q-2022, “Bridges and Bridged Networks.”

Network Segmentation and Micro-Segmentation

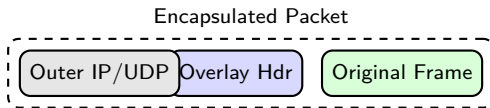
- **VLANs** provide L2 segmentation within a data center
- Additional segmentation strategies:
 - **Firewall rules** between VLAN groups
 - **ACLs** on virtual routers
 - **Micro-segmentation**: per-VM or per-workload firewall policies

Goal: **least privilege** – each VM should only reach what it strictly needs.

Micro-segmentation is a key concept in modern data center security (zero trust networking).

Overlay Networks – Concept

- An **overlay network** is a virtual network built **on top of** the physical one
- Uses **encapsulation**: wrap virtual frames inside physical packets



- The physical network only sees the **outer headers**
- The virtual (tenant) traffic is hidden inside → **full isolation**

VXLAN – Overview

- **VXLAN** (Virtual eXtensible LAN): most widely used overlay protocol
- Encapsulates L2 frames in **UDP packets** over the physical network
- Uses a **24-bit VNI** (VXLAN Network Identifier)
 - $2^{24} = \mathbf{16 \text{ million}}$ virtual networks (vs 4,096 VLANs!)
- Endpoints: **VTEPs** (VXLAN Tunnel Endpoints)
 - Encapsulate at source, decapsulate at destination

Source: RFC 7348, “Virtual eXtensible Local Area Network (VXLAN),” 2014.

VXLAN – How It Works



- VM A sends a frame → VTEP 1 wraps it in UDP with a VNI
- Travels over the physical network as a regular UDP packet
- VTEP 2 strips outer headers, delivers original frame to VM B
- VMs on the **same VNI** form an isolated L2 domain

Overlay Benefits for Scalability

- **16 million virtual networks** (vs 4,096 VLANs)
- Physical network doesn't need to know about tenants
- VMs can **migrate** across hosts without reconfiguring the underlay
- Decouples **virtual topology** from physical topology
- Foundation for cloud networking:
 - AWS VPC, Azure VNet, Google VPC all use overlays internally

Source: RFC 8926, “Geneve: Generic Network Virtualization Encapsulation,” 2020.

Outline

- 1 Introduction to Network Virtualization
- 2 Virtualization Concepts
- 3 Virtual Network Infrastructure
- 4 Network Isolation and Multi-Tenancy
- 5 References**
- 6 Session Summary

References

- ❶ Popek & Goldberg, “Formal Requirements for Virtualizable Third Generation Architectures,” *CACM*, vol. 17, no. 7, 1974.
- ❷ Barham et al., “Xen and the Art of Virtualization,” *SOSP*, 2003.
- ❸ VMware, “Understanding Full Virtualization, Paravirtualization, and Hardware Assist,” 2007.
- ❹ Pfaff et al., “The Design and Implementation of Open vSwitch,” *NSDI*, 2015.
- ❺ IEEE 802.1Q-2022, “Bridges and Bridged Networks.”
- ❻ IETF RFC 7348, “Virtual eXtensible Local Area Network (VXLAN),” 2014.
- ❼ IETF RFC 8926, “Geneve: Generic Network Virtualization Encapsulation,” 2020.
- ❽ Docker, Inc., “Docker Overview,” docs.docker.com.
- ❾ NIST SP 800-125, “Guide to Security for Full Virtualization Technologies,” 2011.
- ❿ Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.

Outline

- 1 Introduction to Network Virtualization
- 2 Virtualization Concepts
- 3 Virtual Network Infrastructure
- 4 Network Isolation and Multi-Tenancy
- 5 References
- 6 Session Summary**

Key Takeaways

- 1 Physical infrastructure is **rigid, underutilized, and slow to scale**
- 2 **Hypervisors** (Type 1 and 2) enable multiple VMs on one host
- 3 **Containers** are lighter than VMs but share the kernel (more in Block 5)
- 4 Virtual networks mirror physical ones: **vNICs, vSwitches, vRouters**
- 5 **Multi-tenancy** requires isolation → VLANs, overlays, micro-segmentation
- 6 **VXLAN** extends VLANs from 4K to 16M networks using encapsulation

Discussion

*You manage a data center with 5,000 tenants.
Each tenant wants their own isolated network.
Why can't you use VLANs alone?
What would you use instead, and why?*

Hints: VLAN ID limit, overlapping IP ranges, VM mobility.